

Up next in Hands-on Labs →

Lab 05: The Infrastructure Pipeline



Hands-on lab · 55 min · Env: See book overview · Overview & prereqs · Download PDF ·

Reset checks

CI/CD ENGINEERING

Shared Libraries: Versioned Groovy
Logic

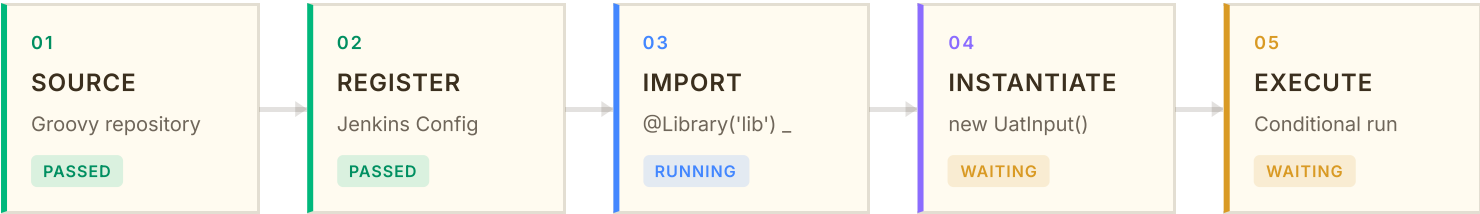
CODE REUSABILITY
GLOBAL PIPELINE LIBRARIES
CONDITIONAL EXECUTION

Nabawy



01 / Reusable Pipeline Logic

As pipeline complexity grows, duplicating Groovy logic across dozens of Jenkinsfiles creates a maintenance nightmare. A Shared Library allows you to store versioned Groovy classes in a standard Git repository and import them into any pipeline.



WHAT

A Git repository containing Groovy code that Jenkins dynamically downloads and makes available to pipelines via an import statement.

WHY

To enforce DRY (Don't Repeat Yourself) principles in CI/CD. Updates to shared logic propagate to all pipelines without modifying individual Jenkinsfiles.

WHEN

When multiple pipelines share common logic: deployment steps, approval gates, metric parsing, or custom build wrappers.

WHAT CHANGES

Pipelines become thinner. Complex business logic is abstracted into testable Groovy classes, moving Jenkins configuration closer to standard software engineering.

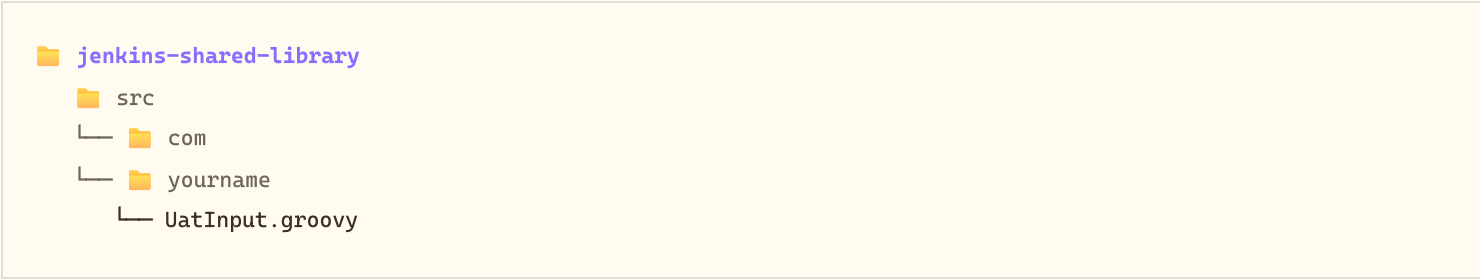
WHY THIS EXISTS

Without shared libraries, organizations end up with hundreds of bloated Jenkinsfiles that are impossible to update. A shared library transforms CI/CD logic into a versioned, testable software dependency.

02 / Repository Architecture

Jenkins expects a specific directory structure for shared libraries. Groovy classes must reside in the `src/` directory, following standard Java package naming conventions.

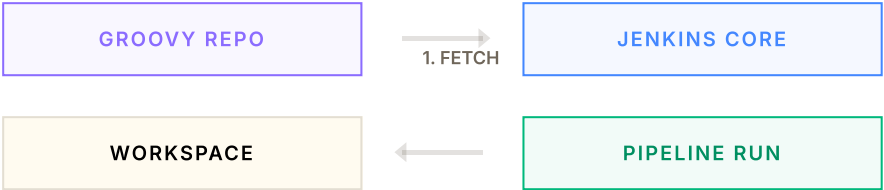
DIRECTORY STRUCTURE



CLASS DEFINITION: UATINPUT.GROOVY

```
package com.yourname
class UatInput {
    def buildIsUatApproved() {
    def file = new File('tools/approved.txt')
    return file.exists()
    }
}
```

EXECUTION ARCHITECTURE

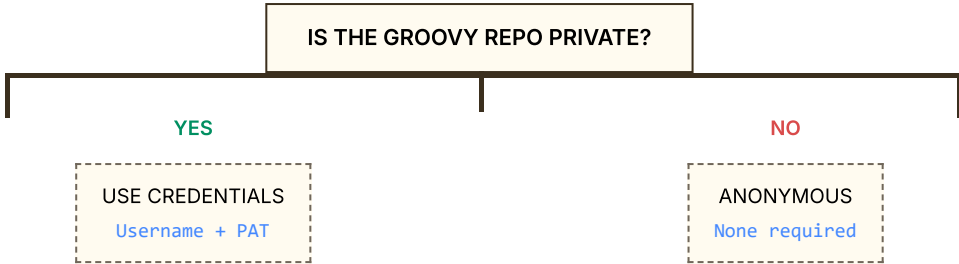


GOTCHA

The package path in the file (`package com.yourname`) must exactly match the folder structure (`src/com/yourname/`). A mismatch will result in a `ClassNotFoundException` during pipeline execution.

03 / Jenkins Registration

Before a pipeline can import the library, Jenkins must be aware of its existence. Register the library globally so it is available to all jobs without per-job configuration.



SYSTEM CONFIGURATION

```
Manage Jenkins → Configure System
Global Pipeline Libraries → Add
> Library Name: shared-library
> Default Version: master
> Retrieval Method: Modern SCM
> SCM: GitHub
> Project Repository: https://github.com/user/repo.git
Click "Validate" to test connection.
```

GOTCHA

By default, shared library scripts run in the Groovy Sandbox. If your custom class uses standard Java I/O (like `new File()`), you must uncheck "Use Groovy Sandbox" in the Pipeline job configuration, or explicitly approve the method signatures in **Manage Jenkins → In-process Script Approval**.

INTERVIEW CHECK

Q: What is the difference between Global Pipeline Libraries and Automatic Pipeline Libraries?

A: Global libraries are available to any pipeline but must be explicitly imported via `@Library`. Automatic libraries are implicitly imported into all pipelines without the annotation, which is convenient but can hide dependencies.

04 / Scripted Pipeline Usage

Import the library into a scripted pipeline using the `@Library` annotation. Instantiate the class and call its methods to evaluate workspace state.

```
@Library('shared-library') _
import com.yourname.UatInput
node {
  stage('Check Approval') {
    def uatInput = new UatInput()
    def isApproved = uatInput.buildIsUatApproved()
    println "Build is UAT approved: ${isApproved}"
  }
}
```

EXECUTION RESULT (FILE PRESENT)

COMMAND

```
$ touch tools/approved.txt
$ build now
```

RESULT

```
[Pipeline] stage
[Pipeline] {(Check Approval)
Build is UAT approved: true
[Pipeline] }
```

WHY IT MATTERS

The underscore `_` in `@Library('shared-library') _` is required when importing classes. It tells Jenkins to import the library but not automatically create any global variables or scripts, only making the classes available for explicit instantiation.

05 / Declarative & Conditionals

In declarative pipelines, the imported class can be used inside a `when` block to conditionally skip or execute stages based on workspace state.

```
@Library('shared-library') _
import com.yourname.UatInput
def uatInput = new UatInput()
pipeline {
  agent any
  stages {
    stage('Deploy to Prod') {
      when { expression { return uatInput.buildIsUatApproved() } }
      steps { echo 'Deployment approved.' }
    }
  }
}
```

STATE TRANSITION

APPROVED	tools/approved.txt exists	RUN STAGE
UNAPPROVED	tools/approved.txt missing	SKIP STAGE

GOTCHA

The `when` directive is evaluated before the `agent` is allocated for the stage if `beforeAgent true` is not set. If your Groovy class reads files from the workspace, ensure the agent is checked out first, or evaluate the condition inside a `script` block within the `steps`

06 / Lab: End-to-End Validation

Test the shared library logic by toggling the existence of the approval file and observing the pipeline's state transition.

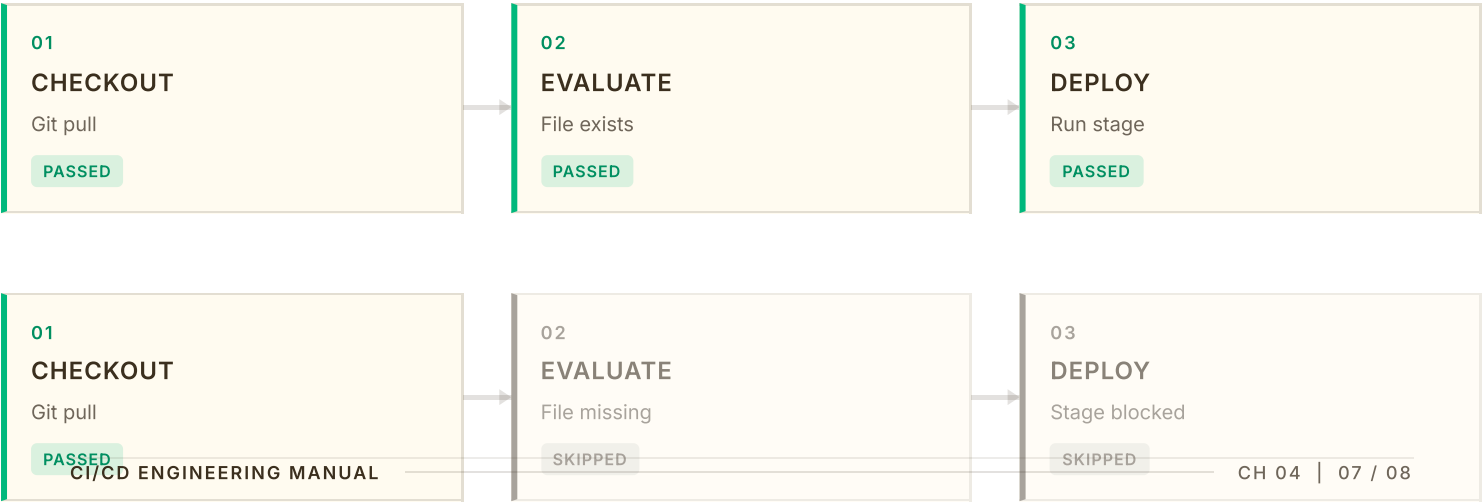
- OBSERVE

The declarative pipeline is configured. We need to verify that the stage execution is strictly controlled by the UatInput class.
- EXECUTE

Run the pipeline with `tools/approved.txt` present. Then, delete the file and run the pipeline again.
- VERIFY

Check the Jenkins build logs. The first run should execute the stage. The second run should show "Stage skipped due to when conditional".
- PROOF

The pipeline dynamically altered its behavior based on workspace state evaluated by external, versioned Groovy code.



07 / System Map & Troubleshooting

Shared libraries introduce external dependencies into the pipeline. Failures usually stem from package path mismatches, sandbox restrictions, or network access to the Git repository.



TROUBLESHOOTING FLOW

ClassNotFoundException	Package path mismatch	Ensure folder structure matches package declaration	
SYMPTOM	ROOT CAUSE	RECOVERY	
Library not found	Typo in @Library name	Match annotation to Global Library name exactly	
Stale logic executing	Caching / wrong branch	Verify Default Version in Jenkins config	

INTERVIEW CHECK

Q: Why disable the Groovy Sandbox for shared libraries?

A: The sandbox limits dangerous operations (like file system access) for untrusted scripts. Shared libraries are considered trusted code. Disabling the sandbox allows the library to perform necessary system operations, though it requires strict review of the library code.

Up next in Hands-on Labs →

Lab 05: The Infrastructure Pipeline

RELATED BOOKS

Lab 05: The Infrastructure Pipeline
60 min · **ADV**
→ Next in path

Lab 03: Multibranch Foundations
45 min · **INT**
→ Previous in path

Lab 01: Automated Integration Foundations
45 min · **INT**
→ Shared tags

Lab 02: Container Delivery
50 min · **INT**
→ Shared tags